

Graph Basics

Kanishk Goel

March 6, 2026

1 Basics

Graph - vertices/nodes V , $|V| = n$ and edges E , $|E| = m$

$G = (V, E)$, undirected graph - each edge corresponds to an unordered pair $\{v, w\}$ of vertices, which are called the endpoints of the edges.

directed graph - each edge (v, w) is an ordered pair, v (tail) and w (head)

Sparse graphs - n vertices and $\mathcal{O}(n \log n)$ Dense graphs - n vertices and $\Omega(\frac{n^2}{\log n})$

2 Adjacency Lists

1. array with graph's vertices and edges
2. $\forall e, e \in E$, a pointer to each of its own endpoints
3. $\forall v, v \in V$, a pointer to each of the incident edge.

for directed graphs, each edge tracks which endpt is tail and head.

memory requirement - $\Theta(m + n)$

3 Adjacency Matrix

$$A_{ij} = \begin{cases} 1 & \text{if edge}(i, j) \text{ belongs to } E \\ 0 & \text{otherwise.} \end{cases}$$

1 bit maintained for each pair of vertices indicating presence of edge. It is easy to extend this:

1. Parallel Edges: A_{ij} defined as the number of edges with given endpoints i and j .
2. Weighted Graphs: If each edge has weight w_{ij} representing cost/distance, then $A_{ij} = w_{ij}$
3. Directed Graphs: Simply asymmetric adjacency matrix.
4. Undirected Graphs: A symmetric adjacency matrix.

memory requirement - $\Theta(n^2)$ Adjacency matrix is an efficient way of storing dense graphs but not sparse graphs.

4 Cut of Graphs

A cut of a graph (V, E) is a partition of V into two non-empty sets A and B .

Crossing Edges The edges connecting one node from A and one node from B .

Problem: Minimum Cut Problem
Input: An undirected graph $G = (V, E)$ with parallel edges allowed. Output: Compute a cut with fewest number of crossing edges